MAHATMA GANDHI MISSION'S

COLLEGE OF COMPUTER SCIENCE & INFORMATION TECHNOLOGY

(Affiliated to University of Mumbai)

Kamothe, Navi-Mumbai – 410209

Batch: - 2023-2026

Certificate

This is to certify that Mr. /Ms. _____Roll No. _____ and Seat No._____ of T.Y.B.Sc.IT (Sem-VI) Class, has performed the experiment in the subject_____ As per guidelines of University of Mumbai.

_____ _____

INTERNAL EXAMINER **EXTERNAL EXAMINER**

_____ _____

COLLEGE SEAL **PRINCIPAL**

Index

Sr. No.	Title	Dates	Remark
0.	Introduction to Android, Introduction to Android Studio IDE, Application Fundamentals: Creating a project, Android Components, Activities, Services, Content Providers, Broadcast Receivers, Interface Overview, Creating Android Virtual Device, USB Debugging mode, Android Application Overview.		
1.	Programming Android Resources (Color, Theme, String, Drawable, Dimensions, Image).		
2.	Programming Android Activity Life Cycle, Methods, Multiple Activities Life Cycle of fragments and Multiple Fragments.		
3.	Programming Android Coordinate, Linear, Relative, Table, Absolute, Frame, List View and Grid View Layouts.		
4.	Programming Android UI Elements AppBar, Fragment, UI Components.		
5.	Programming Menus, Dialog, Dialog Fragments.		
6.	Programs on Intents, Events, Listener and Adapters.		
7.	Programs on Services, Notification and Broadcast Receivers.		
8.	**a.** Database Programming with SQLite. **b.** Programming Network Communication and Services (JSON).		
9.	Programming threads, handler and asynchronized programs.		
10.	a. Programming Media API and Telephone API. b. Programming Security and Permissions.		

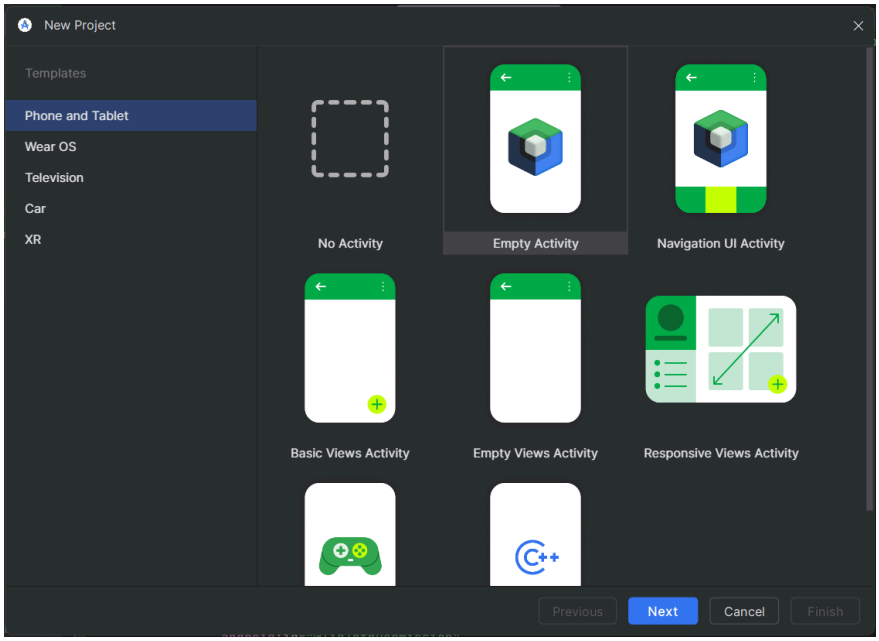
Practical 0

Introduction to Android, Introduction to Android Studio IDE, Application Fundamentals: Creating a project, Android Components, Activities, Services, Content Providers, Broadcast Receivers, Interface Overview, Creating Android Virtual Device, USB Debugging mode, Android Application Overview.

- Head over to the link to get the Android Studio executable or zip file . https://developer.android.com/studio
- After the downloading has finished, open the file from downloads and run it .

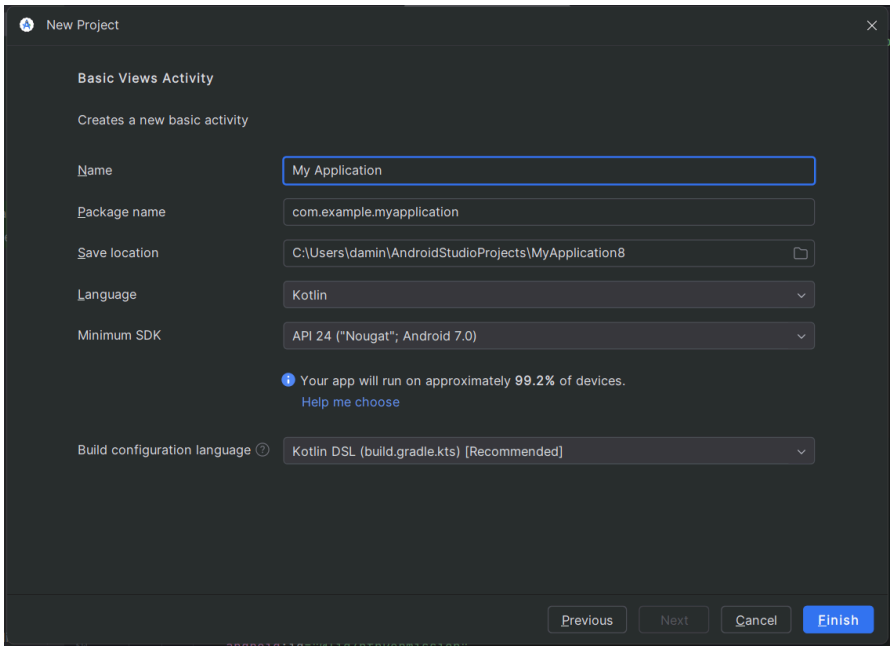
To create a project, follow these steps:

- Install the latest version of Android Studio.
- In the Welcome to Android Studio window, click Start a new Android Studio project.
- If you have a project already opened, select File > New > New Project.
- In the Choose your project window, select Empty Activity and click Next.
- In the Configure your project window, complete the following:



1. Enter "**My Application**" in the Name field.
2. Enter "**com.example.myapplication**" in the Package name field.
3. If you'd like to place the project in a different folder, change its Save location.
4. Select either **Java or Kotlin** from the Language drop-down menu.

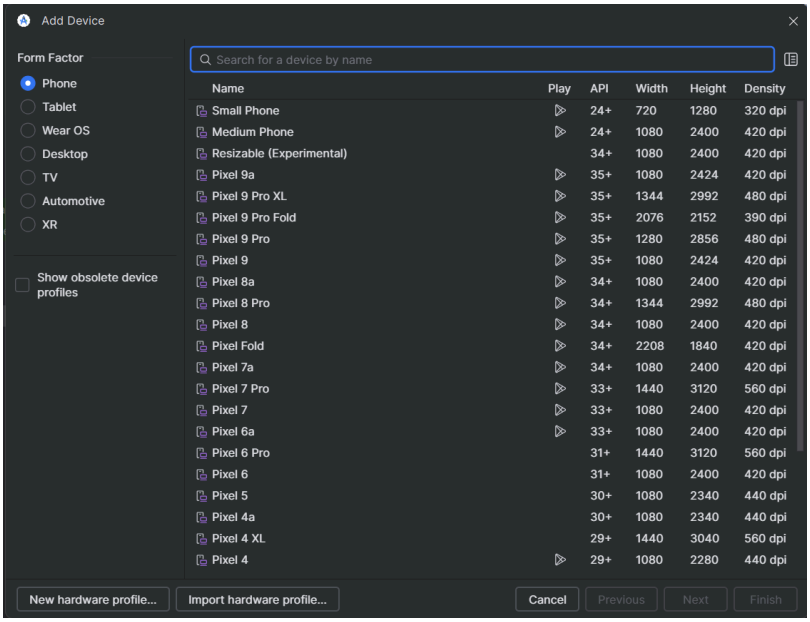
- Click Finish.
- After some processing time, the Android Studio main window appears

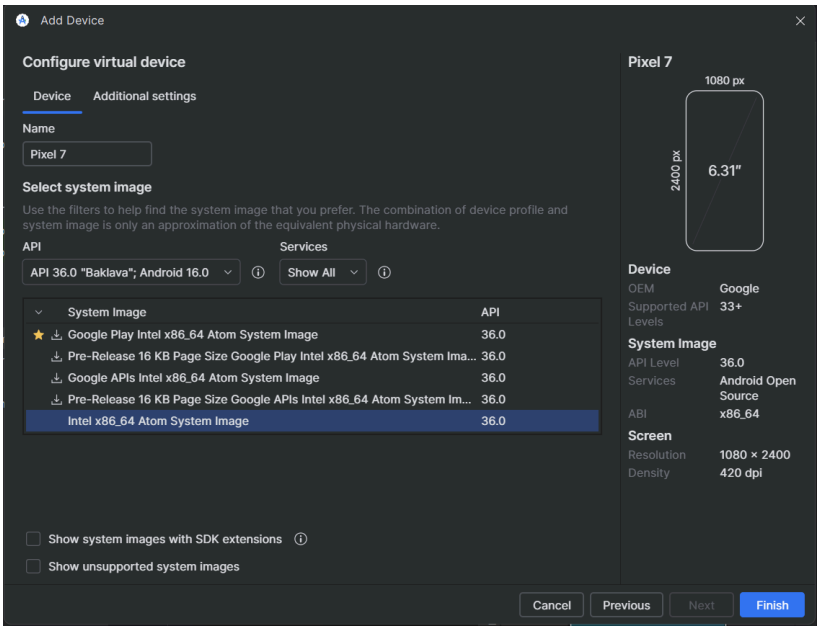


- After the project is created, there are 2 files, MainActivity.java and activity_main.xml
- Go to activity_main.xml and select Design View
- In Design View, change the layout to LinearLayout(Vertical) select
- Add TextView, and change the text to “Hello World!”
- Click on Run and select the AVD already created(if not created, first
- create the AVD)
- Output screen should show “Hello World”

To create a new AVD:

- Open the AVD Manager by clicking Device Manager.
- Click Add a new Device, click create visual device.
- Select a hardware profile, and then click Next.
- Select the system image for a particular API level, and then click Next.
- Change AVD properties as needed, and then click Finish.

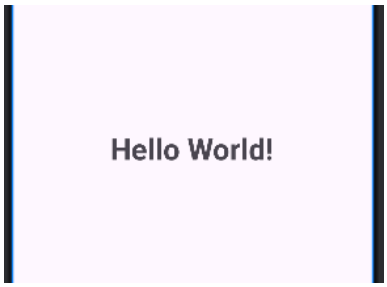




MainActivity.xml

```
package com.example.myapplication
import androidx.appcompat.app.AppCompatActivity
import android.os.Bundle

class MainActivity : AppCompatActivity() {

    override fun onCreate(savedInstanceState: Bundle?) {
```

```
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)

        // Set Toolbar Title
        supportActionBar?.title = "Hello World"
    }
}
```



Activity_main.xml

```
<TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Hello World!"
        android:textSize="34sp"
        android:textStyle="bold"
        android:layout_gravity="center" />
```

Android Components :

Android Components are the basic building blocks of an Android application that help the app perform different tasks like UI display, background processing, data sharing, and communication.

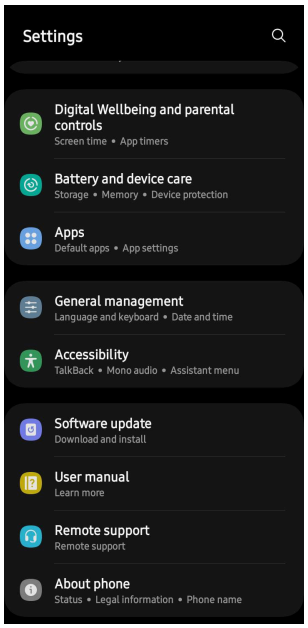
Main 4 Components:

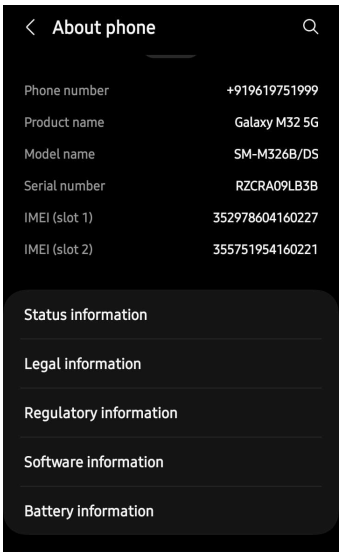
1. **Activity :** Activity is a component that represents a single screen with a user interface in an Android app.
2. **Service :** Service is a component that runs in the background to perform long-running operations without user interaction.
3. **Content Provider :** Content Provider is used to store, manage, and share app data between different applications.
4. **Broadcast Receiver :** Broadcast Receiver is a component that listens and responds to system-wide broadcast messages.

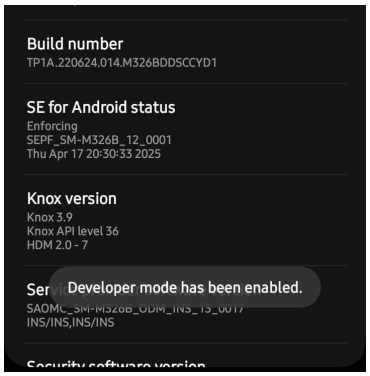
Interface Overview : Interface in Android refers to the graphical layout and interaction elements (like buttons, text fields, menus) that allow users to interact with the app.

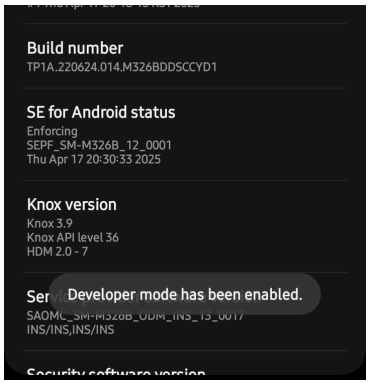
1. USB Debugging Steps :
2. Open Settings on your mobile phone.
3. Scroll down and tap About Phone.
4. Select Software Information.

5. Find the Build Number option.
6. Tap on Build Number 3–7 times continuously.
7. A message will appear: “Developer Mode has been enabled.”
8. Go back to Settings → Developer Options.
9. In Developer Options, you can enable:
    - USB Debugging
    - Wireless Debugging









Practical 1

Programming Resources Android Resources: (Color, Theme, String, Drawable, Dimension, Image),

1. **Open color.xml file**

```
<?xml version="1.0" encoding="utf-8"?>
<resources>
<color name="colorPrimary">#008577</color>
<color name="colorPrimaryDark">#00574B</color>
<color name="colorAccent">#D81B60</color>

// Define new colors here using hexadecimal color codes.

<color name="Red">#FF0000</color>
<color name="Blue">#0000FF</color>
<color name="Pink">#FF00FF</color>
<color name="Cyan">#00FFFF</color>
<color name="Grey">#AABBFF</color>
</resources>
```

Now go to activity_main.xml and type the following code:

- Drag and drop LinearLayout(Vertical)
- From Layout tab in Palette window
- Drag and drop four TextView from Text tab in palette window
- The default Code of one TextView is as follows

```
<LinearLayout
android:layout_width="409dp"
android:layout_height="729dp"
android:orientation="vertical"
tools:layout_editor_absoluteX="1dp"
tools:layout_editor_absoluteY="1dp">

<TextView
android:id="@+id/textView"
android:layout_width="match_parent"
android:layout_height="56dp"
android:text="TextView"
android:background="@color/Red"
/>
```

```
<TextView
android:id="@+id/textView2"
android:layout_width="match_parent"
android:layout_height="52dp"
android:text="TextView"
android:background="@color/Grey"
/>

<TextView
android:id="@+id/textView3"
android:layout_width="match_parent"
android:layout_height="59dp"
android:text="TextView"
android:background="@color/Blue"
/>

<TextView
android:id="@+id/textView4"
android:layout_width="match_parent"
android:layout_height="51dp"
android:text="TextView"
android:background="@color/Cyan"
/>

</LinearLayout>
</androidx.constraintlayout.widget.ConstraintLayout>
```

2. **Defining Theme Property.**

- Defining new theme properties in styles.xml
- Create a new project and go to:
- ProjectName>App>src>main>res>values>styles.xml

styles.xml

```
<resources xmlns:tools="http://schemas.android.com/tools">
  <!-- Base application theme. -->
  <style name="Base.Theme.Pract2" parent="Theme.Material3.DayNight.NoActionBar">

    <!-- Customize your light theme here. -->
    <!-- Customize your theme here. →

      <item name="colorPrimary">@color/colorPrimary</item>
      <item name="colorPrimaryDark">@color/colorPrimaryDark</item>
      <item name="colorAccent">@color/colorAccent</item>
      <item name="android:background">#000000</item>
```

```
        <item name="android:textColor">#FFFFAA</item>
    </style>
</resources>
```

3. Defining string property

- Defining paragraph and header property in strings.xml
- Create a new project and go to:
- ProjectName>App>src>main>res>values>strings.xml

strings.xml

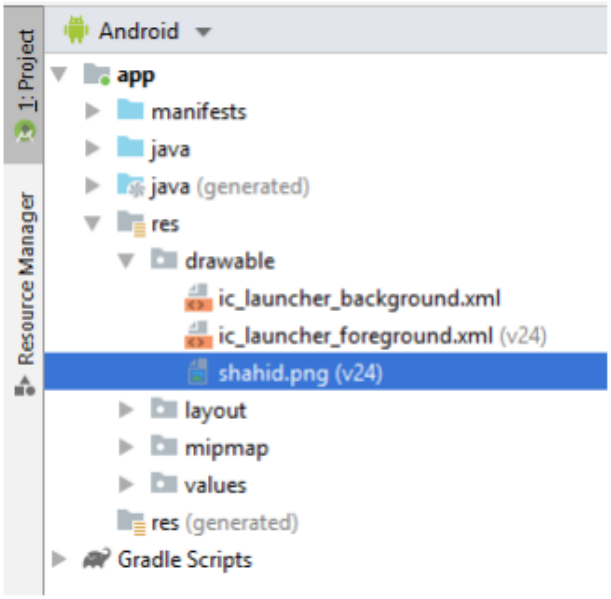
```
<resources>
<string name="app_name">Prof. Damini Sharma</string>
<string name="Heading">Programming Resources</string>
<string name="Description"></string>
</resources>
```

Now go to activity_main.xml and type the following code:

```
<TextView
android:id="@+id/textView"
android:layout_width="match_parent"
android:layout_height="56dp"
android:background="@color/Red"
android:textSize="24sp"
android:text="@string/Heading"
/>

<TextView
android:id="@+id/textView2"
android:layout_width="match_parent"
android:layout_height="wrap_content"
android:background="@color/Grey"
android:textSize="24sp"
android:text="@string/Description"
/>
```

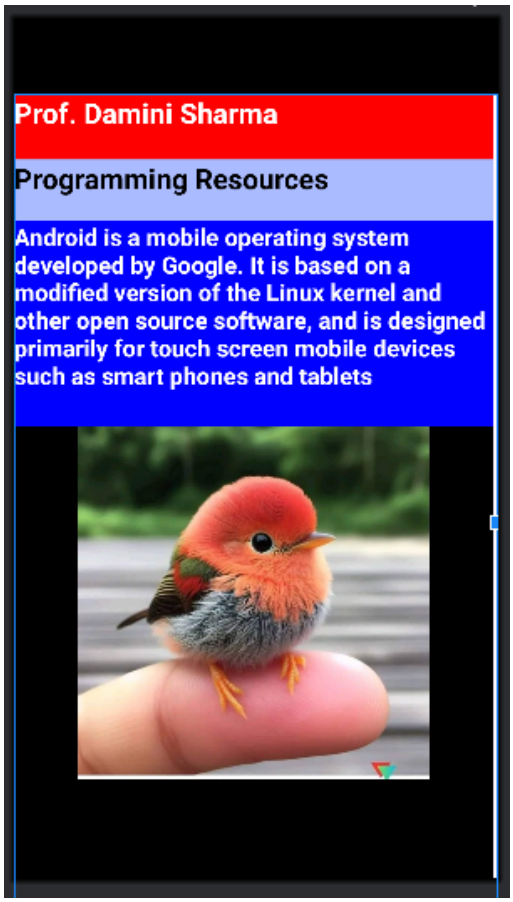
4. Adding images and dimensions

- Adding Images to Application created
- For adding a new image files, do the following:
- ProjectName>App>src>main>res>drawable>Right-click and paste the images that are copied



Add a new dim.xml file and write the following code in dimens.xml

```
<?xml version="1.0" encoding="utf-8" ?>
<resources>
<dimen name="textview_height">35dp</dimen>
<dimen name="textview_width">150dp</dimen>
<dimen name="font_size">26sp</dimen>
</resources>
```

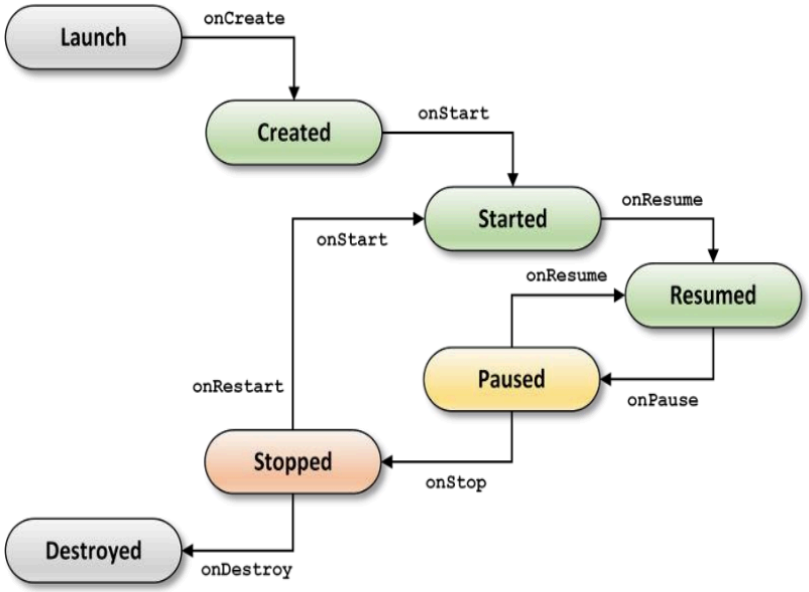


Now go to activity_main.xml and type the following code:

```
<LinearLayout
android:layout_width="409dp"
android:layout_height="729dp"
android:orientation="vertical"
tools:layout_editor_absoluteX="1dp"
tools:layout_editor_absoluteY="1dp">

<ImageView
android:id="@+id/imageView"
android:layout_width="match_parent"
android:layout_height="wrap_content"
android:src="@drawable/shahid"
 />
</LinearLayout>
</androidx.constraintlayout.widget.ConstraintLayout>
```

Practical 2

Programming Activities and fragments:-
(Activity Life Cycle, Activity methods, Multiple Activities, Life Cycle of fragments and multiple fragments.)



1. **New Project Create**

- Android Studio open
- New Project → Empty Views Activity
- Project Name: ActivityLifecycleApp
- Language: Kotlin
- Finish

Kotlin code for ActivityLifeCycle:

```
package com.example.activitylifecycleapp
import android.os.Bundle
import android.util.Log
import androidx.appcompat.app.AppCompatActivity

class MainActivity : AppCompatActivity() {

    val TAG = "Lifecycle"

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
```

```
        setContentView(R.layout.activity_main)
        Log.d(TAG, "onCreate called")
    }
    override fun onStart() {
        super.onStart()
        Log.d(TAG, "onStart called")
    }
    override fun onResume() {
        super.onResume()
        Log.d(TAG, "onResume called")
    }
    override fun onPause() {
        super.onPause()
        Log.d(TAG, "onPause called")
    }
    override fun onStop() {
        super.onStop()
        Log.d(TAG, "onStop called")
    }
    override fun onRestart() {
        super.onRestart()
        Log.d(TAG, "onRestart called")
    }
    override fun onDestroy() {
        super.onDestroy()
        Log.d(TAG, "onDestroy called")
    }
}
```

Activity_main.xml code:

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:gravity="center"
    android:orientation="vertical">

    <TextView
        android:text="Activity Life Cycle Demo"
```

```
        android:textSize="20sp"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"/>
</LinearLayout>
```

App RUN (EXECUTE)

AVD (Emulator) create

Tools → Device Manager → Create Virtual Device

Green Run ▶ button

App emulator will open

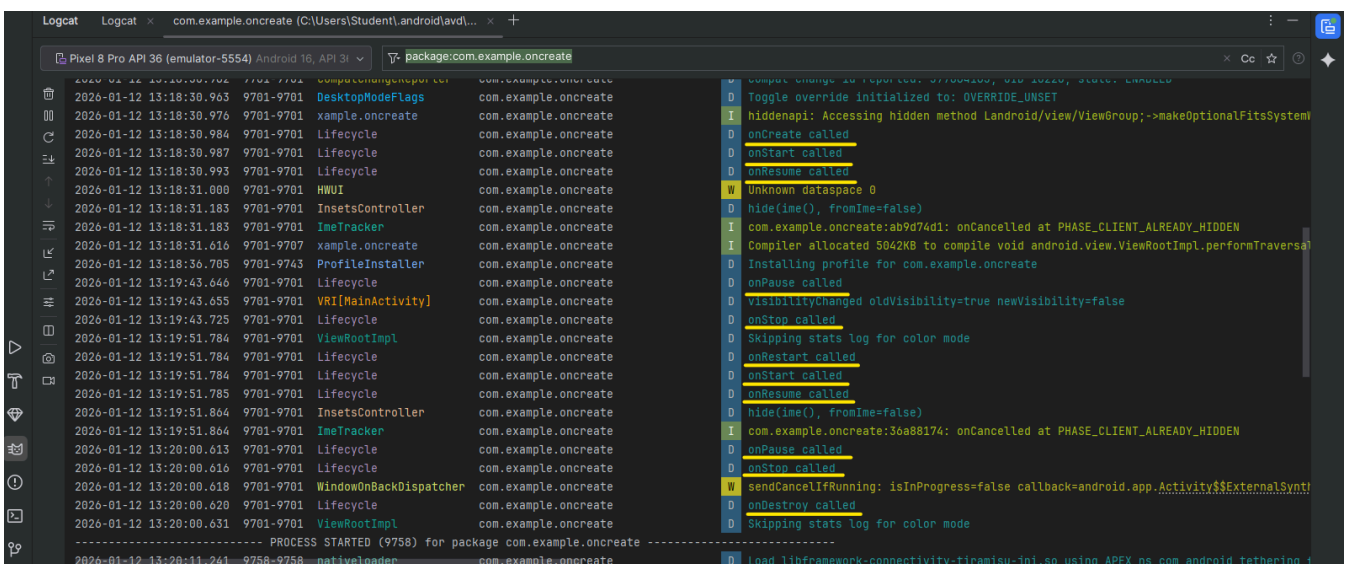
- **Logcat open (MOST IMPORTANT)**
- **Bottom Logcat tab open**
- **Search : Lifecycle**

Output:

D/Lifecycle: onCreate called

D/Lifecycle: onStart called

D/Lifecycle: onResume called

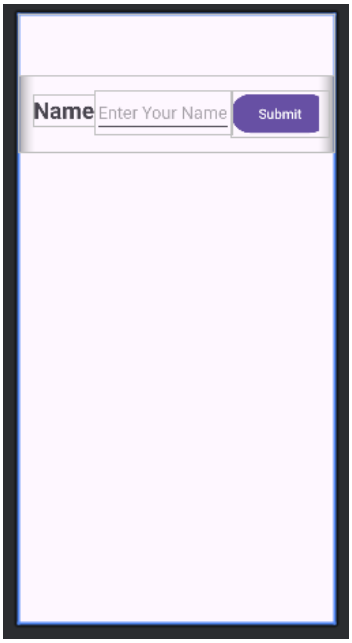


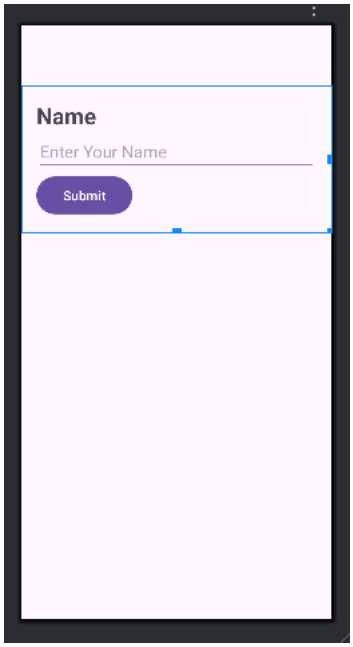
Practical 3

Programs related to different Layouts
(Linear, Relative, Table, ListView, GridView, Absolute)

1. Linear Layout

```
<?xml version="1.0" encoding="utf-8"?>
  <LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="horizontal"  —---------------- >    (horizontal / vertical)
    android:padding="16dp">

    <TextView
      android:id="@+id/textTitle"
      android:layout_width="wrap_content"
      android:layout_height="wrap_content"
      android:text="LinearLayout Demo"
      android:textSize="24sp"
      android:textStyle="bold" />

    <EditText
      android:id="@+id/editName"
      android:layout_width="match_parent"
      android:layout_height="wrap_content"
      android:hint="Enter your name" />

    <Button
      android:id="@+id/btnSubmit"
      android:layout_width="wrap_content"
      android:layout_height="wrap_content"
      android:text="Submit" />

  </LinearLayout>
</com.google.android.material.appbar.AppBarLayout>
</androidx.coordinatorlayout.widget.CoordinatorLayout>
```





2. Absolute Layout

- AbsoluteLayout was deprecated long ago (API level 3) — Android recommends using newer layouts like FrameLayout, RelativeLayout, or ConstraintLayout instead.

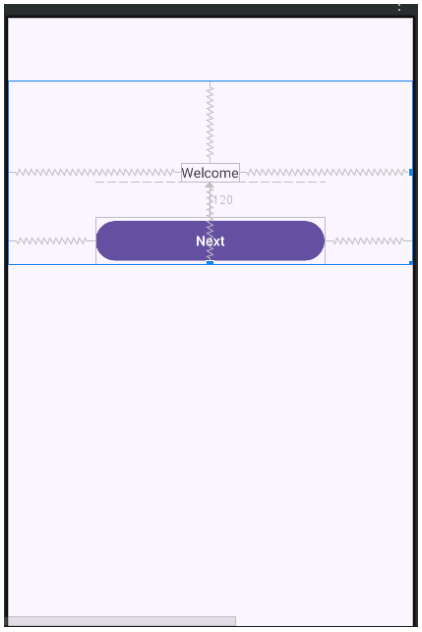
- The whole idea of positioning views with exact layout_x and layout_y doesn't scale well on different screens (small phones vs big tablets), so Android doesn't want you to use it.

- Because Android Studio expects modern layouts, it highlights this code as error/warning even though older Android versions used to support it.

Old code:

```
<?xml version="1.0" encoding="utf-8"?>
<AbsoluteLayout
   xmlns:android="http://schemas.android.com/apk/res/android"
   android:layout_width="match_parent"
   android:layout_height="match_parent">

   <Button
      android:id="@+id/button_first"
      android:layout_width="wrap_content"
      android:layout_height="wrap_content"
      android:text="Click"
      android:layout_x="150dp"
      android:layout_y="500dp" />

</AbsoluteLayout>
```

3. Relative Layout

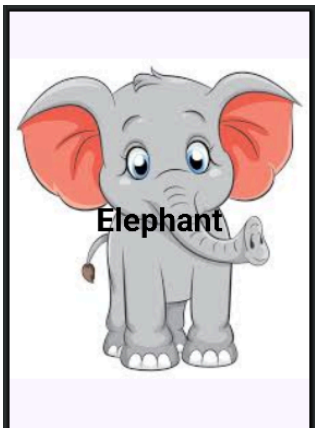



```
<RelativeLayout
   android:layout_width="match_parent"
   android:layout_height="match_parent">

   <TextView
      android:id="@+id/text"
      android:layout_width="wrap_content"
      android:layout_height="wrap_content"
      android:text="Welcome"
      android:layout_centerInParent="true"/>

   <Button
      android:layout_width="wrap_content"
      android:layout_height="wrap_content"
      android:text="Next"
      android:layout_below="@id/text"
      android:layout_centerHorizontal="true"
      android:layout_marginTop="12dp"/>
```

```
</RelativeLayout>
</com.google.android.material.appbar.AppBarLayout>
</androidx.coordinatorlayout.widget.CoordinatorLayout>
```

4. Frame Layout

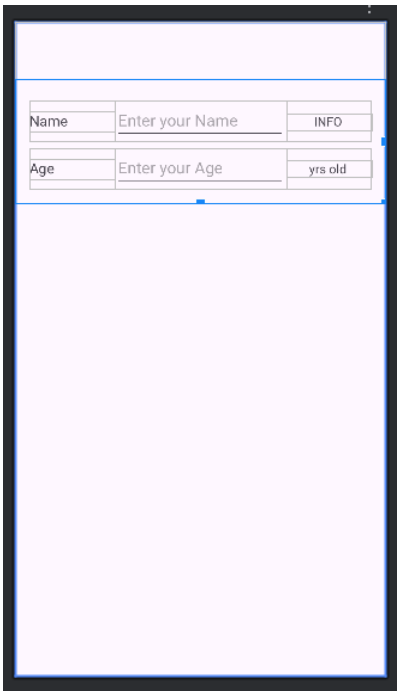
```
<FrameLayout
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context=".MainActivity">

    <ImageView
        android:id="@+id/image"
        android:layout_width="match_parent"
        android:layout_height="match_parent"
        android:scaleType="centerCrop"
        android:src="@drawable/img" />

    <TextView
        android:id="@+id/text2"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="@string/next"
        android:textSize="44sp"
        android:textStyle="bold"
        android:textColor="@android:color/white"
        android:layout_gravity="center"/>
</FrameLayout>
```



5. Table Layout

```
<TableLayout
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:stretchColumns="1,2"
    android:padding="16dp">

    <!-- Row 1: Name, TextInput, Extra Info -->
    <TableRow
        android:layout_marginTop="8dp">
```

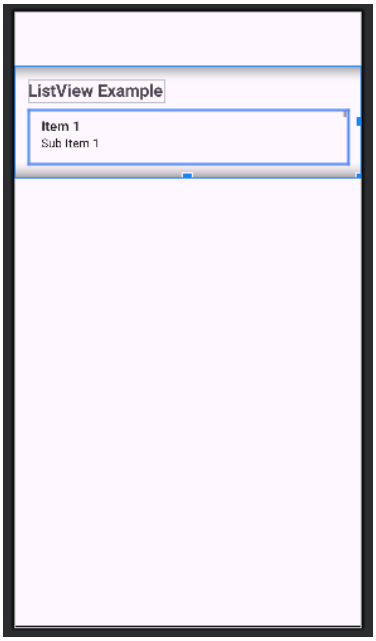
```
    <TextView
        android:layout_width="0dp"
        android:layout_height="wrap_content"
        android:layout_weight="1"
        android:text="@string/name"
        android:textSize="16sp"/>

    <EditText
        android:layout_width="0dp"
        android:layout_height="wrap_content"
        android:layout_weight="2"
        android:hint="@string/hint"/>

    <TextView
        android:layout_width="0dp"
        android:layout_height="wrap_content"
        android:layout_weight="1"
        android:text="@string/info"
        android:gravity="center"/>
</TableRow>

<!-- Row 2: Age, TextInput, Unit/Extra -->
<TableRow
    android:layout_marginTop="8dp">

    <TextView
        android:layout_width="0dp"
        android:layout_height="wrap_content"
        android:layout_weight="1"
        android:text="@string/age"
        android:textSize="16sp"/>

    <EditText
        android:layout_width="0dp"
        android:layout_height="wrap_content"
        android:layout_weight="2"
        android:hint="@string/hint2"
        android:inputType="number"/>
```



```
            <TextView
                android:layout_width="0dp"
                android:layout_height="wrap_content"
                android:layout_weight="1"
                android:text="@string/years"
                android:gravity="center"/>
        </TableRow>

        <!-- You can add more rows the same way -->

    </TableLayout>

  </com.google.android.material.appbar.AppBarLayout>
</androidx.coordinatorlayout.widget.CoordinatorLayout>
```

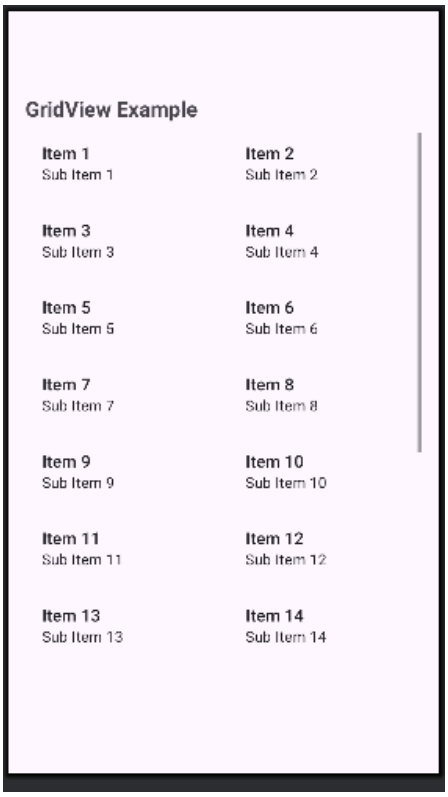
6. List View

```
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
  android:layout_width="match_parent"
  android:layout_height="match_parent"
  android:orientation="vertical"
  android:padding="16dp">

  <TextView
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="ListView Example"
    android:textSize="20sp"
    android:textStyle="bold"
    android:layout_marginBottom="10dp"/>

  <ListView
    android:id="@+id/listView"
    android:layout_width="match_parent"
    android:layout_height="match_parent"/>
</LinearLayout>
</com.google.android.material.appbar.AppBarLayout>
</androidx.coordinatorlayout.widget.CoordinatorLayout>
```



7. Grid View

```
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:padding="16dp">

    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="GridView Example"
        android:textSize="20sp"
        android:textStyle="bold"
        android:layout_marginBottom="10dp"/>

    <GridView
        android:id="@+id/gridView"
        android:layout_width="match_parent"
        android:layout_height="match_parent"
        android:numColumns="2"
        android:horizontalSpacing="10dp"
        android:verticalSpacing="10dp"
        android:gravity="center"/>
</LinearLayout>
</com.google.android.material.appbar.AppBarLayout>
</androidx.coordinatorlayout.widget.CoordinatorLayout>
```



Practical 4

Programming UI elements (AppBar, Fragments, UI Components)

MainActivity.kt

```
package com.example.myapplication
import android.os.Bundle
import androidx.appcompat.app.AppCompatActivity
import androidx.appcompat.widget.Toolbar

class MainActivity : AppCompatActivity() {

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)

        val toolbar = findViewById<Toolbar>(R.id.toolbar)
        setSupportActionBar(toolbar)

        // Load First Fragment
        supportFragmentManager.beginTransaction()
            .replace(R.id.fragmentContainer, FirstFragment())
            .commit()
    }
}
```

```
java/
└── com.example.uielements
    ├── MainActivity.kt
    ├── FirstFragment.kt
    └── SecondFragment.kt

res/
├── layout/
│   ├── activity_main.xml
│   ├── fragment_first.xml
│   └── fragment_second.xml
└── menu/
    └── menu_main.xml
```

activity_main.xml

```
    <!-- AppBar -->
    <com.google.android.material.appbar.AppBarLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content">

        <androidx.appcompat.widget.Toolbar
            android:id="@+id/toolbar"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:background="@color/black"
            app:title="UI Elements App"
            app:titleTextColor="@android:color/white"/>
```

```
    </com.google.android.material.appbar.AppBarLayout>

    <!-- Fragment Container -->
    <FrameLayout
        android:id="@+id/fragmentContainer"
        android:layout_width="match_parent"
        android:layout_height="match_parent"
        android:layout_marginTop="?attr/actionBarSize"/>

</androidx.coordinatorlayout.widget.CoordinatorLayout>
```

FirstFragment.kt

```
package com.example.myapplication
import android.os.Bundle
import android.view.LayoutInflater
import android.view.View
import android.view.ViewGroup
import android.widget.Button
import androidx.fragment.app.Fragment

class FirstFragment : Fragment() {

    override fun onCreateView(
        inflater: LayoutInflater,
        container: ViewGroup?,
        savedInstanceState: Bundle?
    ): View {

        val view = inflater.inflate(R.layout.fragment_first, container, false)

        val button = view.findViewById<Button>(R.id.btnNext)

        button.setOnClickListener {
            parentFragmentManager.beginTransaction()
                .replace(R.id.fragmentContainer, SecondFragment())
                .addToBackStack(null)
                .commit()
        }
```

```
        return view
    }
}
```

fragment_first.xml

```
<LinearLayout>
    <TextView
        android:text="First Fragment"
        android:textSize="20sp"
        android:textStyle="bold"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"/>

    <EditText
        android:id="@+id/editTextName"
        android:hint="Enter your name"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"/>

    <Button
        android:id="@+id/btnNext"
        android:text="Go to Second Fragment"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"/>
</LinearLayout>
```

SecondFragment.kt

```
package com.example.myapplication
import android.os.Bundle
import android.view.LayoutInflater
import android.view.View
import android.view.ViewGroup
import androidx.fragment.app.Fragment

class SecondFragment : Fragment() {

    override fun onCreateView(
```

```
        inflater: LayoutInflater,
        container: ViewGroup?,
        savedInstanceState: Bundle?
    ): View {
        return inflater.inflate(R.layout.fragment_second, container, false)
    }
}
```

fragment_second.xml

```
<LinearLayout>
    <TextView
        android:id="@+id/text"
        android:text="Second Fragment"
        android:textSize="20sp"
        android:textStyle="bold"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"/>

    <CheckBox
        android:id="@+id/checkBox"
        android:layout_width="103dp"
        android:layout_height="wrap_content"
        android:text="I agree" />

    <RadioGroup
        android:layout_width="103dp"
        android:layout_height="wrap_content">
        <RadioButton
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:text="Male" />
        <RadioButton
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:text="Female" />
    </RadioGroup>
</LinearLayout>
```

Practical 5

Programming menus, dialog, dialog fragments

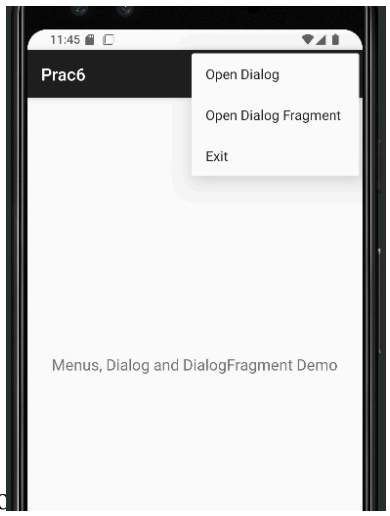
Main_activity.kt

```
package com.example.myapplication
import android.os.Bundle
import android.view.Menu
import android.view.MenuItem
import android.widget.Toast
import androidx.appcompat.app.AlertDialog
import androidx.appcompat.app.AppCompatActivity

class MainActivity : AppCompatActivity() {

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)
    }
    override fun onCreateOptionsMenu(menu: Menu?): Boolean {
        menuInflater.inflate(R.menu.menu_main, menu)
        return true
    }
    override fun onOptionsItemSelected(item: MenuItem): Boolean {
        when (item.itemId) {
            R.id.menu_dialog -> {
                showAlertDialog()
                return true
            }
            R.id.menu_dialog_fragment -> {
                val dialog = MyDialogFragment()
                dialog.show(supportFragmentManager, "MyDialog")
                return true
            }
            R.id.menu_exit -> {
                finish()
                return true
            }
        }
        return super.onOptionsItemSelected(item)
```

```
    }

    // Simple Alert Dialog
    private fun showAlertDialog() {
        val builder = AlertDialog.Builder(this)
        builder.setTitle("Simple Dialog")
        builder.setMessage("This is Alert Dialog example")
        builder.setPositiveButton("OK") { _, _ ->
            Toast.makeText(this, "OK Clicked", Toast.LENGTH_SHORT).show()
        }
        builder.setNegativeButton("Cancel", null)
        builder.show()
    }
}
```

mydialogfragment.kt

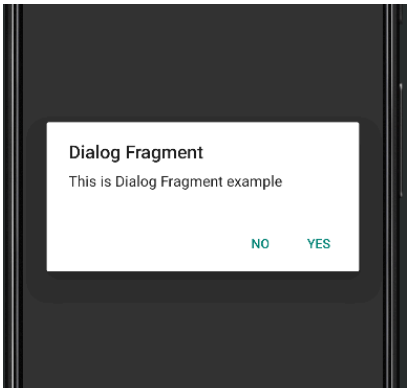
```
package com.example.myapplication
import android.app.Dialog
import android.os.Bundle
import androidx.appcompat.app.AlertDialog
import androidx.fragment.app.DialogFragment

class MyDialogFragment : DialogFragment() {
    override fun onCreateDialog(savedInstanceState: Bundle?): Dialog {
        return AlertDialog.Builder(requireContext())
            .setTitle("Dialog Fragment")
            .setMessage("This is Dialog Fragment example")
            .setPositiveButton("Yes") { _, _ -> }
            .setNegativeButton("No") { _, _ -> }
            .create()
    }
}
```



menu_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<menu xmlns:android="http://schemas.android.com/apk/res/andro
    <item
        android:id="@+id/menu_dialog"
```

```
        android:title="Open Dialog" />
    <item
        android:id="@+id/menu_dialog_fragment"
        android:title="Open Dialog Fragment" />
    <item
        android:id="@+id/menu_exit"
        android:title="Exit" />
</menu>
```



Activity_main.xml

```
    <TextView
        android:text="Menus, Dialog and DialogFragment Demo"
        android:textSize="18sp"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"/>
</LinearLayout>
```

values/themes

```
<resources xmlns:tools="http://schemas.android.com/tools">
    <!-- Base application theme. -->
    <style name="Base.Theme.Prac_6" parent="Theme.AppCompat.DayNight.DarkActionBar">
        <!-- Customize your light theme here. -->
        <!-- <item name="colorPrimary">@color/my_light_primary</item> -->
    </style>
    <style name="Theme.Prac_6" parent="Base.Theme.Prac_6" />
</resources>
```

v23/themes

```
<resources xmlns:tools="http://schemas.android.com/tools">
    <style name="Theme.Prac_6" parent="Base.Theme.Prac_6">
        <!-- Transparent system bars for edge-to-edge. -->
        <item name="android:navigationBarColor">@android:color/transparent</item>
        <item name="android:statusBarColor">@android:color/transparent</item>
        <item name="android:windowLightStatusBar">?attr/isLightTheme</item>
    </style>
</resources>
```

Practical 6

Programs on Intents, Events Listeners and Adapters

1. **Event Listeners**

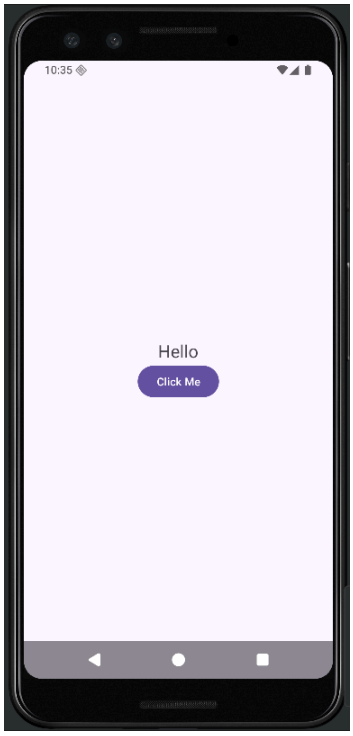
Main_Activity.kt

```
package com.example.myapplication
import android.os.Bundle
import android.widget.Button
import android.widget.TextView
import androidx.appcompat.app.AppCompatActivity

class MainActivity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)

        val textView = findViewById<TextView>(R.id.textView)
        val button = findViewById<Button>(R.id.button)

        button.setOnClickListener {
            textView.text = "Button Clicked!"
        }
    }
}
```

Activity_main.xml

```
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:orientation="vertical"
    android:gravity="center"
    android:layout_width="match_parent"
    android:layout_height="match_parent">

    <TextView
        android:id="@+id/textView"
        android:text="Hello"
        android:textSize="22sp"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"/>
```

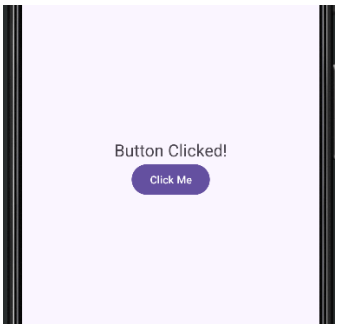
```
        <Button
            android:id="@+id/button"
            android:text="Click Me"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"/>
    </LinearLayout>
</androidx.coordinatorlayout.widget.CoordinatorLayout>
```



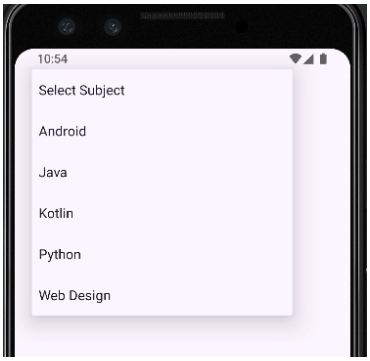
2. Adapters

Main_Activity.kt

```
package com.example.myapplication
import android.os.Bundle
import android.widget.ArrayAdapter
import android.widget.Spinner
import android.widget.Toast
import androidx.appcompat.app.AppCompatActivity

class MainActivity : AppCompatActivity() {

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)

        val spinner = findViewById<Spinner>(R.id.spinner)
        val subjects = arrayOf(
            "Select Subject",
            "Android",
            "Java",
            "Kotlin",
            "Python",
            "Web Design"
        )
        val adapter = ArrayAdapter(
            this,
            android.R.layout.simple_spinner_item,
            subjects
        )
        adapter.setDropDownViewResource(
```

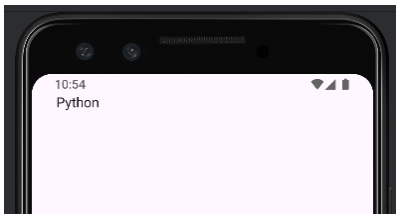


```
        android.R.layout.simple_spinner_dropdown_item
    )

    spinner.adapter = adapter
  }
}
```



Activity_main.xml

```
  <LinearLayout
    android:orientation="vertical"
    android:padding="80dp"
    android:layout_width="match_parent"
    android:layout_height="match_parent">

    <Spinner
      android:id="@+id/spinner"
      android:layout_width="match_parent"
      android:layout_height="wrap_content"/>
  </LinearLayout>
</androidx.coordinatorlayout.widget.CoordinatorLayout>
```



3. **Intent (Implicit Intent)**

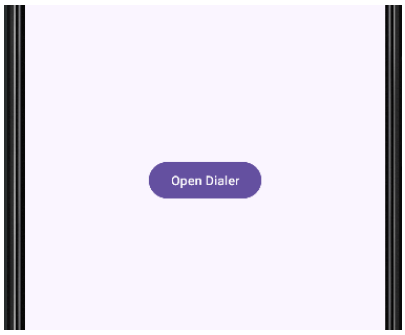
Main_Activity.kt

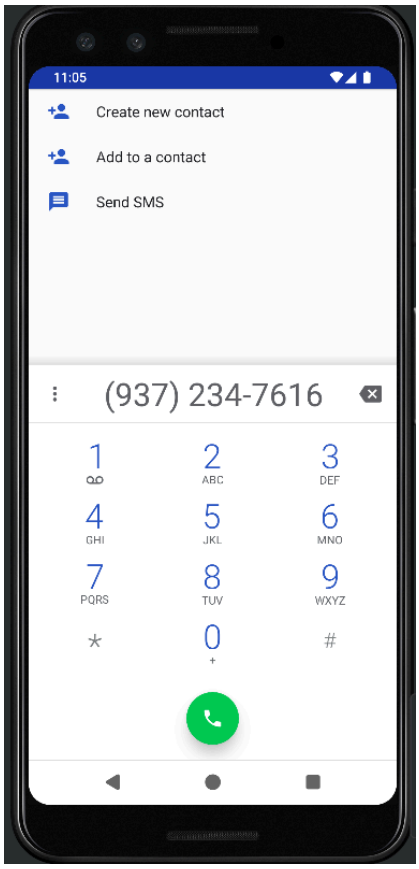
```
package com.example.myapplication

import android.content.Intent
import android.net.Uri
import android.os.Bundle
import android.widget.Button
import androidx.appcompat.app.AppCompatActivity

class MainActivity : AppCompatActivity() {

  override fun onCreate(savedInstanceState: Bundle?) {
    super.onCreate(savedInstanceState)
```

```
        setContentView(R.layout.activity_main)

        val btnCall = findViewById<Button>(R.id.btnCall)

        btnCall.setOnClickListener {
            val intent = Intent(
                Intent.ACTION_DIAL,
                Uri.parse("tel:9372347616")
            )
            startActivity(intent)
        }
    }
}
```

Activity_main.xml

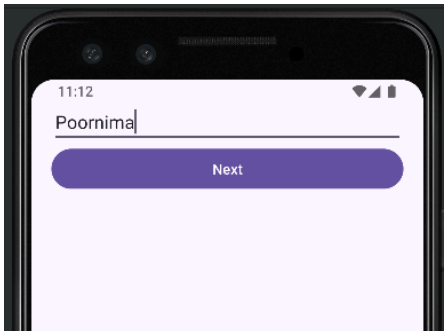
```
    <LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
        android:orientation="vertical"
        android:gravity="center"
        android:layout_width="match_parent"
        android:layout_height="match_parent">

        <Button
            android:id="@+id/btnCall"
            android:text="Open Dialer"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"/>
    </LinearLayout>
</androidx.coordinatorlayout.widget.CoordinatorLayout>
```



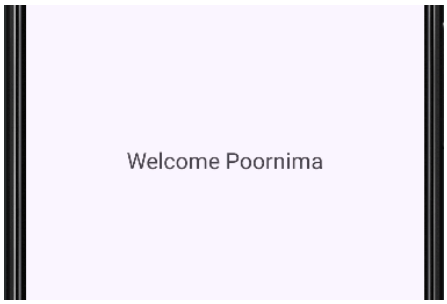


4. **Intent (Explicit Intent)**

MainActivity.kt




```
package com.example.myapplication
import android.content.Intent
import android.os.Bundle
import android.widget.Button
import android.widget.EditText
import androidx.appcompat.app.AppCompatActivity

class MainActivity : AppCompatActivity() {

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)

        val editText = findViewById<EditText>(R.id.editText)
        val btnNext = findViewById<Button>(R.id.btnNext)

        btnNext.setOnClickListener {
            val intent = Intent(this, SecondActivity::class.java)
            intent.putExtra("name", editText.text.toString())
            startActivity(intent)
        }
    }
}
```



SecondActivity.kt

```
package com.example.myapplication
import android.os.Bundle
import android.widget.TextView
import androidx.appcompat.app.AppCompatActivity

class SecondActivity : AppCompatActivity() {

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_second)

        val textView = findViewById<TextView>(R.id.textView)
```

```
        val name = intent.getStringExtra("name")
        textView.text = "Welcome $name"
    }
}
```

Activity_main.xml

```
    <LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
        android:orientation="vertical"
        android:padding="20dp"
        android:layout_width="match_parent"
        android:layout_height="match_parent">

        <EditText
            android:id="@+id/editText"
            android:hint="Enter Name"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"/>

        <Button
            android:id="@+id/btnNext"
            android:text="Next"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"/>
    </LinearLayout>

</androidx.coordinatorlayout.widget.CoordinatorLayout>
```

Activity_second.xml

```
<LinearLayout
    android:gravity="center"
    android:layout_width="match_parent"
    android:layout_height="match_parent">

    <TextView
        android:id="@+id/textView"
        android:textSize="22sp"
        android:layout_width="wrap_content"
```

```
        android:layout_height="wrap_content"/>
</LinearLayout>
```

Manifest.xml

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools">

    <application
        android:allowBackup="true"
        android:dataExtractionRules="@xml/data_extraction_rules"
        android:fullBackupContent="@xml/backup_rules"
        android:icon="@mipmap/ic_launcher"
        android:label="@string/app_name"
        android:roundIcon="@mipmap/ic_launcher_round"
        android:supportsRtl="true"
        android:theme="@style/Theme.MyApplication">
        <activity android:name=".SecondActivity" />
        <activity android:name=".MainActivity"
            android:exported="true">
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />
                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>
    </application>
</manifest>
```

Practical 7

Programs on Services, notification and broadcast receivers

1. **Android Services:** Services are commands which are used by kotlin in functions to execute the task. They are :IntentService,onStartCommand(),onHandleIntent() etc.

Main_activity.kt

```
package com.example.prac8
import android.content.Intent
import android.os.Bundle
import android.widget.Button
import androidx.appcompat.app.AppCompatActivity

class MainActivity : AppCompatActivity() {

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)

        val btnStart = findViewById<Button>(R.id.btnStart)
        val btnStop = findViewById<Button>(R.id.btnStop)

        btnStart.setOnClickListener {
            val intent = Intent(this, MyService::class.java)
            startService(intent)
        }
        btnStop.setOnClickListener {
            val intent = Intent(this, MyService::class.java)
            stopService(intent)
        }
    }
}
```

Activity.xml

```
    <LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
        android:layout_width="match_parent"
        android:layout_height="match_parent"
```

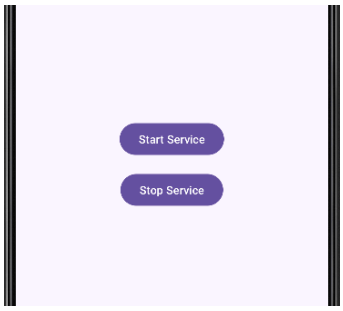
```
        android:gravity="center"
        android:orientation="vertical">

        <Button
            android:id="@+id/btnStart"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="Start Service" />

        <Button
            android:id="@+id/btnStop"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="Stop Service"
            android:layout_marginTop="16dp"/>
    </LinearLayout>
</androidx.coordinatorlayout.widget.CoordinatorLayout>
```

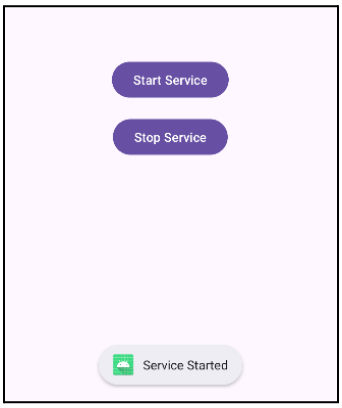
Manifest.xml

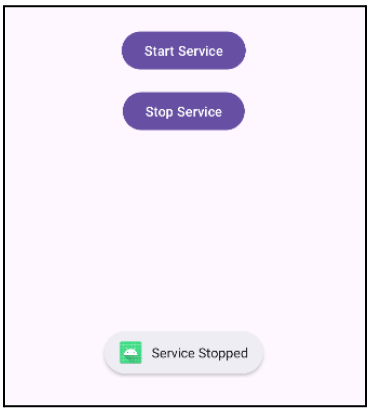
```
    <application
        android:allowBackup="true"
        android:dataExtractionRules="@xml/data_extraction_rules"
        android:fullBackupContent="@xml/backup_rules"
        android:icon="@mipmap/ic_launcher"
        android:label="@string/app_name"
        android:roundIcon="@mipmap/ic_launcher_round"
        android:supportsRtl="true"
        android:theme="@style/Theme.Prac8">
        <activity
            android:name=".MainActivity"
            android:exported="true">
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />
                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>
        <service android:name=".MyService" />
    </application>
</manifest>
```

MyService.kt

```
package com.example.prac8
import android.app.Service
import android.content.Intent
import android.os.IBinder
import android.widget.Toast

class MyService : Service() {

    override fun onCreate() {
        super.onCreate()
        Toast.makeText(this, "Service Created", Toast.LENGTH_SHORT).show()
    }
    override fun onStartCommand(intent: Intent?, flags: Int, startId: Int): Int {
        Toast.makeText(this, "Service Started", Toast.LENGTH_SHORT).show()
        return START_STICKY
    }
    override fun onDestroy() {
        super.onDestroy()
        Toast.makeText(this, "Service Stopped", Toast.LENGTH_SHORT).show()
    }
    override fun onBind(intent: Intent?): IBinder? {
        return null
    }
}
```

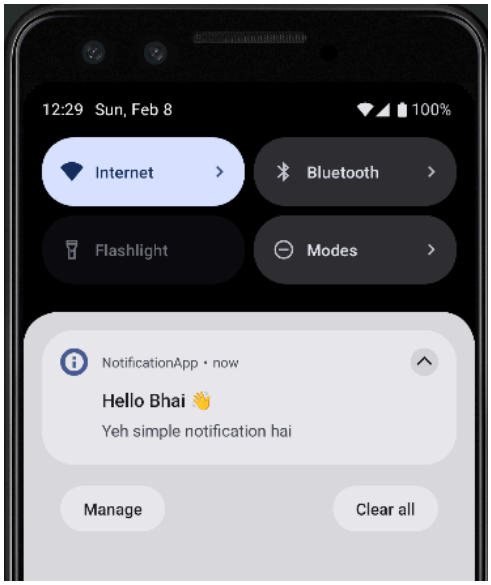






2. **Android Notification:-** Android Notification is a message that is displayed outside the application's user interface to inform the user about an event or update.

MainActivity.kt




```
package com.example.prac8
import android.app.NotificationChannel
import android.app.NotificationManager
import android.content.pm.PackageManager
import android.os.Build
import android.os.Bundle
import android.widget.Button
import androidx.appcompat.app.AppCompatActivity
import androidx.core.app.ActivityCompat
import androidx.core.app.NotificationCompat
import androidx.core.content.ContextCompat

class MainActivity : AppCompatActivity() {

    private val CHANNEL_ID = "simple_channel"
    private val NOTIFICATION_ID = 1

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)

        createNotificationChannel()

        val btn = findViewById<Button>(R.id.btnNotify)
        btn.setOnClickListener {
            showNotification()
        }
    }

    private fun createNotificationChannel() {
        if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.O) {
            val channel = NotificationChannel(
                CHANNEL_ID,
                "Simple Channel",
                NotificationManager.IMPORTANCE_DEFAULT
```

```
        )

        val manager = getSystemService(NotificationManager::class.java)
        manager.createNotificationChannel(channel)
    }
}

private fun showNotification() {

    // Android 13+ permission check
    if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.TIRAMISU) {
        if (ContextCompat.checkSelfPermission(
                this,
                android.Manifest.permission.POST_NOTIFICATIONS
            ) != PackageManager.PERMISSION_GRANTED
        ) {
            ActivityCompat.requestPermissions(
                this,
                arrayOf(android.Manifest.permission.POST_NOTIFICATIONS),
                101
            )
            return
        }
    }

    val notification = NotificationCompat.Builder(this, CHANNEL_ID)
        .setSmallIcon(android.R.drawable.ic_dialog_info)
        .setContentTitle("Hello Bhai 👋")
        .setContentText("Yeh simple notification hai")
        .setAutoCancel(true)
        .build()

    val manager =
        getSystemService(NotificationManager::class.java)

    manager.notify(NOTIFICATION_ID, notification)
}
}
```

Activity.xml

```xml
<LinearLayout
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:gravity="center"
    android:orientation="vertical">

    <Button
        android:id="@+id/btnNotify"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Show Notification"/>
</LinearLayout>
```

Manifest.xml

```xml
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools"
    package="com.example.prac8">

    <!-- Android 13+ ke liye -->
    <uses-permission android:name="android.permission.POST_NOTIFICATIONS"/>

    <application
        android:allowBackup="true"
        android:label="NotificationApp"
        android:theme="@style/Theme.AppCompat.Light.NoActionBar">
        <activity android:name=".MainActivity"
            android:exported="true"
            tools:ignore="MissingClass">
            <intent-filter>
                <action android:name="android.intent.action.MAIN"/>
                <category android:name="android.intent.category.LAUNCHER"/>
            </intent-filter>
        </activity>
    </application>
</manifest>
```

3. **Broadcast Receiver:-** Broadcast Receiver is an Android component that listens for system-wide broadcast messages or events and responds when they occur.

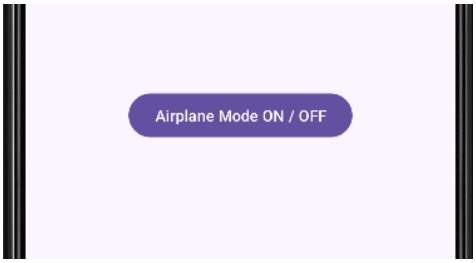
MainActivity.kt

```
package com.example.sqliteexample
import android.content.Intent
import android.os.Bundle
import android.provider.Settings
import android.widget.Button
import androidx.appcompat.app.AppCompatActivity

class MainActivity : AppCompatActivity() {

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)

        val btnAirplane = findViewById<Button>(R.id.btnAirplane)

        btnAirplane.setOnClickListener {
            // Open Airplane Mode Settings
            val intent = Intent(Settings.ACTION_AIRPLANE_MODE_SETTINGS)
            startActivity(intent)
        }
    }
}
```

Activity.xml

```
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
        android:layout_width="match_parent"
        android:layout_height="match_parent"
        android:orientation="vertical"
        android:gravity="center"
        android:padding="20dp">

        <Button
            android:id="@+id/btnAirplane"
            android:layout_width="wrap_content"
```

```
        android:layout_height="wrap_content"
        android:text="Airplane Mode ON / OFF" />
    </LinearLayout>
```



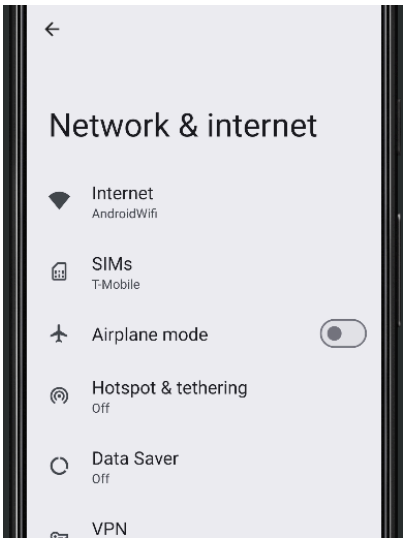
AirplaneModeReceiver.kt

```
package com.example.sqliteexample
import android.content.BroadcastReceiver
import android.content.Context
import android.content.Intent
import android.widget.Toast

class AirplaneModeReceiver : BroadcastReceiver() {

    override fun onReceive(context: Context?, intent: Intent?) {
        val isAirplaneModeOn =
            intent?.getBooleanExtra("state", false) ?: false

        if (isAirplaneModeOn) {
            Toast.makeText(context, "Airplane Mode ON", Toast.LENGTH_SHORT).show()
        } else {
            Toast.makeText(context, "Airplane Mode OFF", Toast.LENGTH_SHORT).show()
        }
    }
}
```

Manifest.xml

```
    <application
        android:allowBackup="true"
        android:dataExtractionRules="@xml/data_extraction_rules"
        android:fullBackupContent="@xml/backup_rules"
        android:icon="@mipmap/ic_launcher"
        android:label="@string/app_name"
        android:roundIcon="@mipmap/ic_launcher_round"
        android:supportsRtl="true"
        android:theme="@style/Theme.SQLiteExample">
        <!-- Main Activity -->
        <activity
            android:name=".MainActivity"
```

```
        android:exported="true">
        <intent-filter>
            <action android:name="android.intent.action.MAIN" />
            <category android:name="android.intent.category.LAUNCHER" />
        </intent-filter>
    </activity>
    <!-- Airplane Mode Broadcast Receiver (IMPORTANT) -->
    <receiver
        android:name=".AirplaneModeReceiver"
        android:enabled="true"
        android:exported="true">
        <intent-filter>
            <action android:name="android.intent.action.AIRPLANE_MODE" />
        </intent-filter>
    </receiver>
  </application>
</manifest>
```



Practical 8

A. Database Programming with SQLite

Database Programming with SQLite in Android:- It means storing, managing and retrieving data locally on the device using SQLite database.

MainActivity.kt

```
package com.example.myapplication
import androidx.appcompat.app.AppCompatActivity
import android.os.Bundle
import android.widget.*

class MainActivity : AppCompatActivity() {

    lateinit var dbHelper: DBHelper
    lateinit var etName: EditText
    lateinit var etAge: EditText
    lateinit var tvResult: TextView

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)

        dbHelper = DBHelper(this)

        etName = findViewById(R.id.etName)
        etAge = findViewById(R.id.etAge)
        tvResult = findViewById(R.id.tvResult)

        val btnSave = findViewById<Button>(R.id.btnSave)
        val btnDelete = findViewById<Button>(R.id.btnDelete)
        val btnShow = findViewById<Button>(R.id.btnShow)

        btnSave.setOnClickListener {
            val name = etName.text.toString()
            val age = etAge.text.toString()

            if (dbHelper.insertData(name, age)) {
                Toast.makeText(this, "Data Saved", Toast.LENGTH_SHORT).show()
```

```
            } else {
                Toast.makeText(this, "Error", Toast.LENGTH_SHORT).show()
            }
        }

        btnDelete.setOnClickListener {
            val name = etName.text.toString()

            if (dbHelper.deleteData(name)) {
                Toast.makeText(this, "Data Deleted", Toast.LENGTH_SHORT).show()
            } else {
                Toast.makeText(this, "Not Found", Toast.LENGTH_SHORT).show()
            }
        }

        btnShow.setOnClickListener {
            val data = dbHelper.getAllData()
            tvResult.text = data
        }
    }
}
```

Activity.xml

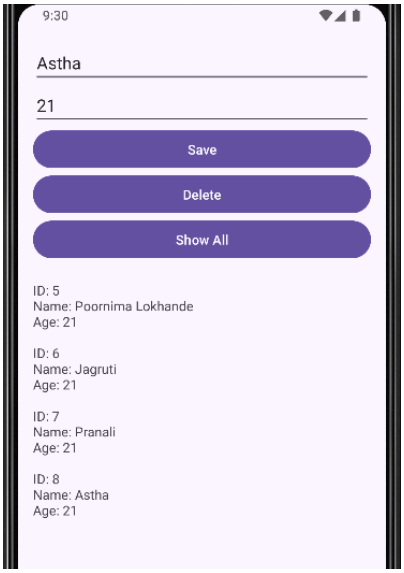
```
<LinearLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:orientation="vertical"
    android:padding="16dp"
    android:layout_width="match_parent"
    android:layout_height="match_parent">

    <EditText
        android:id="@+id/etName"
        android:hint="Enter Name"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"/>

    <EditText
        android:id="@+id/etAge"
        android:hint="Enter Age"
```

```
        android:inputType="number"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"/>

    <Button
        android:id="@+id/btnSave"
        android:text="Save"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"/>

    <Button
        android:id="@+id/btnDelete"
        android:text="Delete"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"/>

    <Button
        android:id="@+id/btnShow"
        android:text="Show All"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"/>

    <TextView
        android:id="@+id/tvResult"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:paddingTop="20dp"/>

  </LinearLayout>
```

DBHelper.kt

```
package com.example.myapplication
import android.content.ContentValues
import android.content.Context
import android.database.sqlite.SQLiteDatabase
import android.database.sqlite.SQLiteOpenHelper

class DBHelper(context: Context) :
  SQLiteOpenHelper(context, "StudentDB", null, 1) {
```

```
    override fun onCreate(db: SQLiteDatabase) {
        db.execSQL(
            "CREATE TABLE Student(id INTEGER PRIMARY KEY AUTOINCREMENT, name
TEXT, age INTEGER)"
        )
    }
    override fun onUpgrade(db: SQLiteDatabase, oldVersion: Int, newVersion: Int) {
        db.execSQL("DROP TABLE IF EXISTS Student")
        onCreate(db)
    }
    fun insertData(name: String, age: String): Boolean {
        val db = this.writableDatabase
        val contentValues = ContentValues()
        contentValues.put("name", name)
        contentValues.put("age", age)
        val result = db.insert("Student", null, contentValues)
        return result != -1L
    }
    fun deleteData(name: String): Boolean {
        val db = this.writableDatabase
        val result = db.delete("Student", "name=?", arrayOf(name))
        return result > 0
    }
    fun getAllData(): String {
        val db = this.readableDatabase
        val cursor = db.rawQuery("SELECT * FROM Student", null)
        var data = ""

        if (cursor.moveToFirst()) {
            do {
                data += "ID: ${cursor.getInt(0)}\n"
                data += "Name: ${cursor.getString(1)}\n"
                data += "Age: ${cursor.getInt(2)}\n\n"
            } while (cursor.moveToNext())
        }
        cursor.close()
        return data
    }
}
```



B. Programming Network Communication and Services (JSON)

JSON API is a web service that provides data in JSON format over the internet.

MainActivity.kt

```
package com.example.sqliteexample
import android.os.Bundle
import android.widget.Button
import android.widget.TextView
import androidx.appcompat.app.AppCompatActivity
import org.json.JSONArray
import java.net.URL
import kotlin.concurrent.thread

class MainActivity : AppCompatActivity() {

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)

        val btnLoad = findViewById<Button>(R.id.btnLoad)
        val tvData = findViewById<TextView>(R.id.tvData)
        btnLoad.setOnClickListener {
            thread {
                try {
                    val response =
                        URL("https://jsonplaceholder.typicode.com/users").readText()

                    val jsonArray = JSONArray(response)
                    val result = StringBuilder()

                    for (i in 0 until jsonArray.length()) {
                        val obj = jsonArray.getJSONObject(i)

                        val name = obj.getString("name")
                        val email = obj.getString("email")

                        result.append("Name: $name\n")
                        result.append("Email: $email\n\n")
```

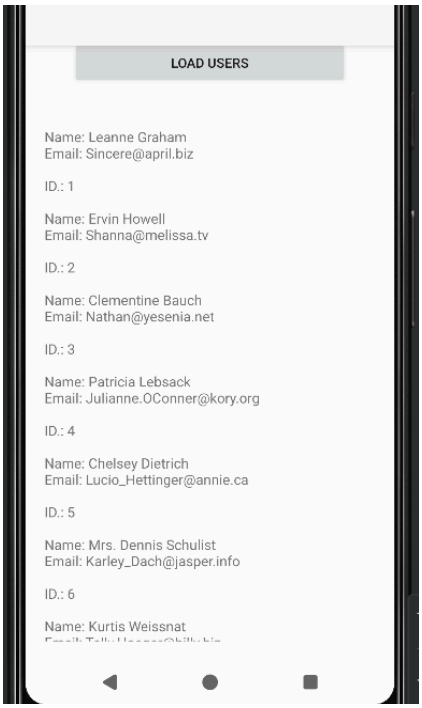
```
                }
                runOnUiThread {
                    tvData.text = result.toString()
                }
            } catch (e: Exception) {
                runOnUiThread {
                    tvData.text = "Error: ${e.message}"
                }
            }
        }
    }
}
```

Activity.xml

```
<LinearLayout
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:padding="20dp">

    <Button
        android:id="@+id/btnLoad"
        android:text="Load Users"
        android:layout_margin="30dp"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"/>

    <TextView
        android:id="@+id/tvData"
        android:text="Data here"
        android:textSize="15sp"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:paddingTop="15dp"/>
</LinearLayout>
```

Manifest.xml

```xml
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    package="com.example.sqliteexample">

    <!-- Internet Permission -->
    <uses-permission android:name="android.permission.INTERNET"/>

    <application
        android:allowBackup="true"
        android:label="JSON Example"
        android:icon="@mipmap/ic_launcher"
        android:roundIcon="@mipmap/ic_launcher_round"
        android:supportsRtl="true"
        android:theme="@style/Theme.AppCompat.Light.NoActionBar"
        android:usesCleartextTraffic="true">
        <activity android:name=".MainActivity"
            android:exported="true">
            <intent-filter>
                <action android:name="android.intent.action.MAIN"/>
                <category android:name="android.intent.category.LAUNCHER"/>
            </intent-filter>
        </activity>
    </application>
</manifest>
```



Practical 9

Programming threads, handler and asynchronized programs

1. **Threads:-** Thread is a lightweight process that runs a task independently in the background without blocking the main UI thread.
2. **UI Threads:-** UI Thread is the main thread of an Android application that handles user interaction and screen updates.

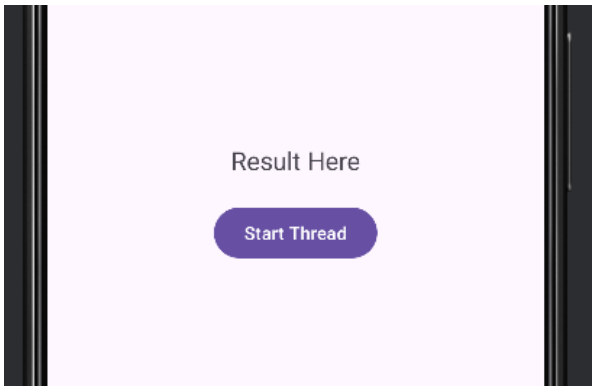
MainActivity.kt

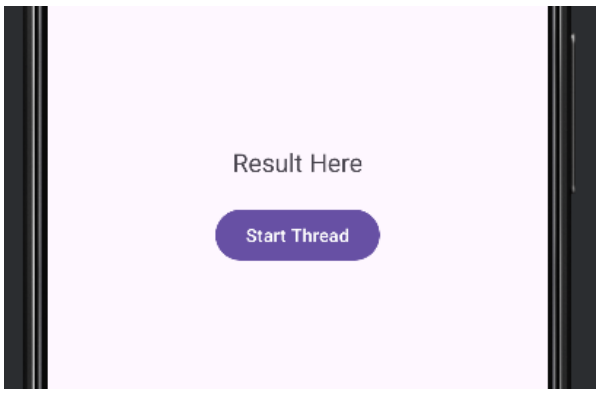
```
package com.example.threadexample
import androidx.appcompat.app.AppCompatActivity
import android.os.Bundle
import android.os.Handler
import android.os.Looper
import android.widget.Button
import android.widget.TextView

class MainActivity : AppCompatActivity() {

    lateinit var tvResult: TextView
    lateinit var btnStart: Button

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)

        tvResult = findViewById(R.id.tvResult)
        btnStart = findViewById(R.id.btnStart)

        btnStart.setOnClickListener {

            Thread {
                // Background work
                Thread.sleep(3000)

                // Switch to UI thread using Handler
                Handler(Looper.getMainLooper()).post {
                    tvResult.text = "Thread Completed!"
                }
```

```
            }.start()
        }
    }
}
```

Activity_main.xml

```
<LinearLayout
    android:orientation="vertical"
    android:gravity="center"
    android:layout_width="match_parent"
    android:layout_height="match_parent">

    <TextView
        android:id="@+id/tvResult"
        android:text="Result Here"
        android:textSize="20sp"
        android:layout_margin="20dp"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"/>

    <Button
        android:id="@+id/btnStart"
        android:text="Start Thread"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"/>
</LinearLayout>
```





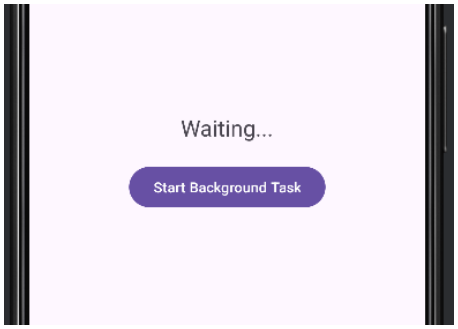
3. **Handler:-** Handler is a class used to send and receive messages between threads, mainly to update the UI from a background thread.

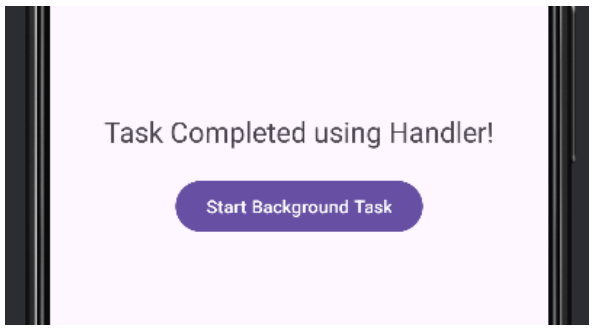
MainActivity.kt

```
package com.example.handlerexample
import androidx.appcompat.app.AppCompatActivity
import android.os.Bundle
import android.os.Handler
import android.os.Looper
import android.widget.Button
import android.widget.TextView

class MainActivity : AppCompatActivity() {

    private lateinit var tvMessage: TextView
    private lateinit var btnStart: Button

    // Handler connected to Main (UI) Thread
    private val handler = Handler(Looper.getMainLooper())

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)

        tvMessage = findViewById(R.id.tvMessage)
        btnStart = findViewById(R.id.btnStart)

        btnStart.setOnClickListener {

            // Background Thread
            Thread {
                Thread.sleep(3000) // Simulate heavy work

                // Send result to UI thread using Handler
                handler.post {
                    tvMessage.text = "Task Completed using Handler!"
                }
            }.start()
        }
    }
}
```

Activity_main.xml

```
<LinearLayout
    android:orientation="vertical"
    android:gravity="center"
    android:layout_width="match_parent"
    android:layout_height="match_parent">

    <TextView
        android:id="@+id/tvMessage"
        android:text="Waiting..."
        android:textSize="22sp"
        android:layout_margin="20dp"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"/>

    <Button
        android:id="@+id/btnStart"
        android:text="Start Background Task"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"/>
</LinearLayout>
```





4. **Asynchronized:-** Asynchronous programming is a technique in which tasks run independently without blocking the main program execution.

Mainactivity.kt

```
package com.example.myapplication
import androidx.appcompat.app.AppCompatActivity
import android.os.Bundle
import android.widget.Button
```

```
import android.widget.TextView
import kotlinx.coroutines.*

class MainActivity : AppCompatActivity() {

    private lateinit var tvResult: TextView
    private lateinit var btnStart: Button

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)

        tvResult = findViewById(R.id.tvResult)
        btnStart = findViewById(R.id.btnStart)

        btnStart.setOnClickListener {

            // Asynchronous task
            CoroutineScope(Dispatchers.IO).launch {

                delay(3000)   // Simulate long operation (3 sec)

                withContext(Dispatchers.Main) {
                    tvResult.text = "Async Task Completed!"
                }
            }
        }
    }
}
```

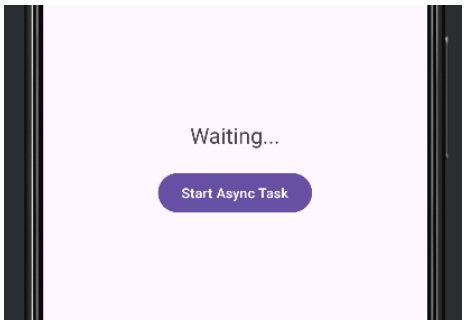
Activity.xml

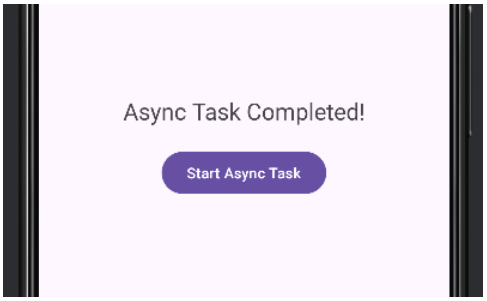
```
<LinearLayout
        android:orientation="vertical"
        android:gravity="center"
        android:layout_width="match_parent"
        android:layout_height="match_parent">

        <TextView
            android:id="@+id/tvResult"
```

```
        android:text="Waiting..."
        android:textSize="22sp"
        android:layout_margin="20dp"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"/>

    <Button
        android:id="@+id/btnStart"
        android:text="Start Async Task"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"/>
</LinearLayout>
```

Build.gradle.kts (Module:app)

```
dependencies {
    .
    .
    .
    implementation("org.jetbrains.kotlinx:kotlinx-coroutines-android:1.7.3")
}
```





Practical 10

A. Programming Media API and Telephone API

- **The Media API in Android is used to handle and play multimedia files such as audio and video. The MediaPlayer class is commonly used to control playback functions like play, pause, and stop.**

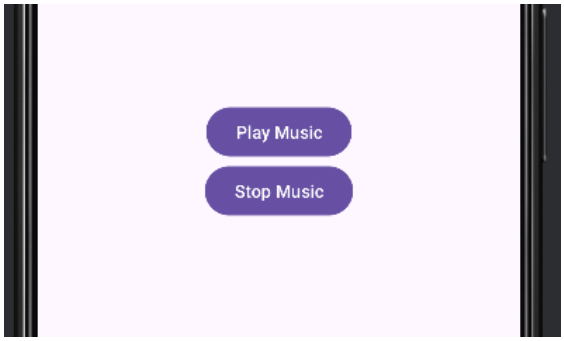
MainActivity .kt

```
package com.example.myapplication
import androidx.appcompat.app.AppCompatActivity
import android.os.Bundle
import android.widget.Button
import android.media.MediaPlayer

class MainActivity : AppCompatActivity() {

    lateinit var mediaPlayer: MediaPlayer

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)

        val btnPlay = findViewById<Button>(R.id.btnPlay)
        val btnStop = findViewById<Button>(R.id.btnStop)

        mediaPlayer = MediaPlayer.create(this, R.raw.song)

        btnPlay.setOnClickListener {
            if (!mediaPlayer.isPlaying) {
                mediaPlayer.start()
            }
        }
        btnStop.setOnClickListener {
            if (mediaPlayer.isPlaying) {
                mediaPlayer.pause()
                mediaPlayer.seekTo(0)   // rewind to start
            }
        }
    }
```

```
    override fun onDestroy() {
        super.onDestroy()
        mediaPlayer.release()   // free memory
    }
}
```

Activity .xml

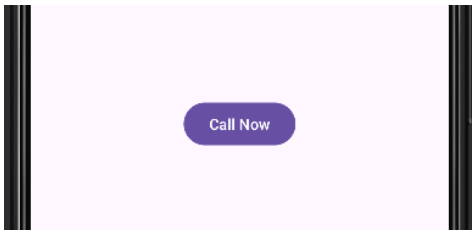
```
    <LinearLayout
        android:orientation="vertical"
        android:gravity="center"
        android:layout_width="match_parent"
        android:layout_height="match_parent">

        <Button
            android:id="@+id/btnPlay"
            android:text="Play Music"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"/>

        <Button
            android:id="@+id/btnStop"
            android:text="Stop Music"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"/>
    </LinearLayout>
```

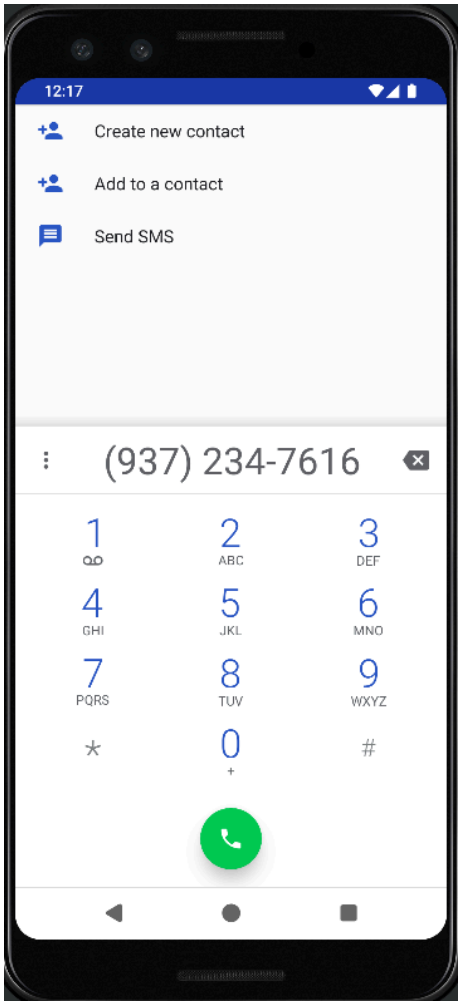


- **The Telephone API in Android is used to perform calling functions. It uses Intent actions such as ACTION_DIAL to open the dialer and ACTION_CALL to make a direct phone call (with required permission).**

MainActivity .kt

```
package com.example.myapplication
import androidx.appcompat.app.AppCompatActivity
import android.os.Bundle
import android.widget.Button
import android.content.Intent
import android.net.Uri

class MainActivity : AppCompatActivity() {

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)

        val btnCall =
findViewById<Button>(R.id.btnCall)

        btnCall.setOnClickListener {

            val intent = Intent(Intent.ACTION_DIAL)
            intent.data = Uri.parse("tel:9372347616")
            startActivity(intent)
        }
    }
}
```





Activity_main.xml

```
<LinearLayout
        android:gravity="center"
        android:orientation="vertical"
        android:layout_width="match_parent"
        android:layout_height="match_parent">
        <Button
            android:id="@+id/btnCall"
            android:text="Call Now"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"/>
    </LinearLayout>
```

B. Programming Security and permissions

Manifest.xml

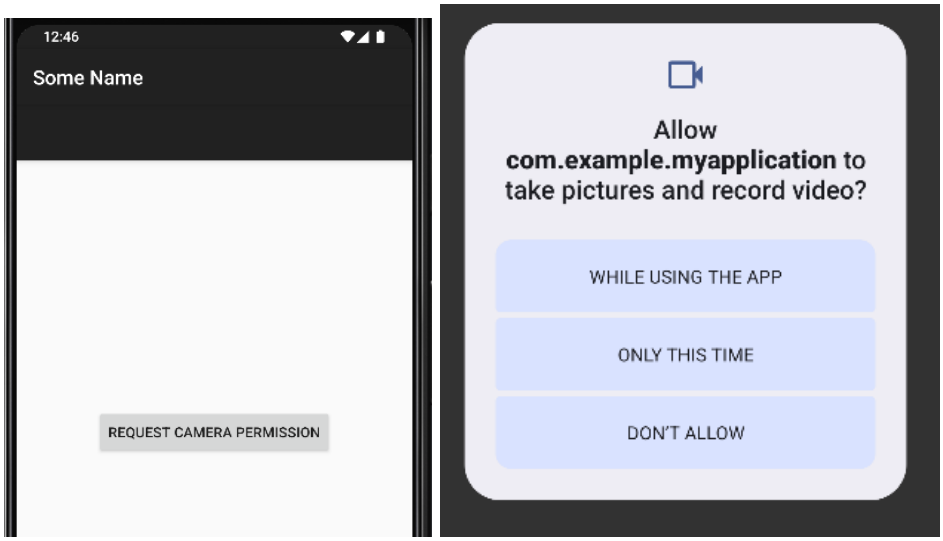
```
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    package="com.example.myapplication">

    <!-- Camera Permission -->
    <uses-feature
        android:name="android.hardware.camera"
        android:required="false" />
    <uses-permission android:name="android.permission.CAMERA"/>

    <application
        android:allowBackup="true"
        android:theme="@style/Theme.AppCompat.Light.DarkActionBar">

        <activity android:name=".MainActivity"
            android:exported="true"
            android:label="Some Name">
            <intent-filter>
                <action android:name="android.intent.action.MAIN"/>
                <category android:name="android.intent.category.LAUNCHER"/>
            </intent-filter>
        </activity>
    </application>
</manifest>
```

MainActivity.kt

```
package com.example.myapplication
import android.Manifest
import android.content.pm.PackageManager
import android.os.Bundle
import android.widget.Button
import android.widget.Toast
import androidx.appcompat.app.AppCompatActivity
import androidx.core.app.ActivityCompat
import androidx.core.content.ContextCompat
```

```
class MainActivity : AppCompatActivity() {

    private val CAMERA_PERMISSION_CODE = 101

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)

        val btnPermission = findViewById<Button>(R.id.btnPermission)

        btnPermission.setOnClickListener {

            if (ContextCompat.checkSelfPermission(
                    this,
                    Manifest.permission.CAMERA
                ) == PackageManager.PERMISSION_GRANTED
            ) {
                Toast.makeText(this, "Permission Already Granted", Toast.LENGTH_SHORT).show()
            } else {

                ActivityCompat.requestPermissions(
                    this,
                    arrayOf(Manifest.permission.CAMERA),
                    CAMERA_PERMISSION_CODE
                )
            }
        }
    }
    override fun onRequestPermissionsResult(
        requestCode: Int,
        permissions: Array<String>,
        grantResults: IntArray
    ) {
        super.onRequestPermissionsResult(requestCode, permissions, grantResults)

        if (requestCode == CAMERA_PERMISSION_CODE) {
            if (grantResults.isNotEmpty() &&
                grantResults[0] == PackageManager.PERMISSION_GRANTED
            ) {
```

```
            Toast.makeText(this, "Camera Permission Granted", Toast.LENGTH_SHORT).show()
        } else {
            Toast.makeText(this, "Camera Permission Denied", Toast.LENGTH_SHORT).show()
        }
    }
  }
}
```

Activity_main.xml

```
<LinearLayout
    android:gravity="center"
    android:orientation="vertical"
    android:layout_width="match_parent"
    android:layout_height="match_parent">

    <Button
      android:id="@+id/btnPermission"
      android:text="Request Camera Permission"
      android:layout_width="wrap_content"
      android:layout_height="wrap_content"/>

  </LinearLayout>
```